Personal Banking Platform

System Design Document

Django · Django REST Framework · React + TypeScript · PostgreSQL · Redis · Celery · Docker · AWS EC2

1. Overview

The Personal Banking Platform is a full-stack, production-deployed web application that simulates the core capabilities of a retail bank and brokerage: customer accounts, real-time money movement, automated monthly statements, and an equities investment engine. The system is deliberately engineered around the two properties that distinguish real financial software from typical CRUD applications — correctness of money movement and security of sensitive data — rather than around raw feature count. Every balance is derived from an immutable double-entry ledger, every monetary value is stored as fixed-precision decimalThe deployed system is seeded to run at a realistic scale: 500+ customer accounts, 10,000+ posted transactions, and an investment engine that executes 1,000+ simulated trades across 50+ ticker symbols. All figures are produced by a deterministic seed script, so the metrics are reproducible on any environment rather than hand-written.

The project pursues three goals: **(1)** demonstrate end-to-end ownership from data model to TLS-terminated cloud deployment; **(2)** model financial invariants that hold under concurrent access; and **(3)** showcase security patterns — envelope encryption, multi-factor authentication, and append-only auditing — drawn from real payment-systems practice.

2. System Architecture

The system is structured as a modular monolith. The Django project is decomposed into focused, well-bounded apps — accounts, ledger, transfers, investing, statements, and security — served behind a single deployable. This keeps operational complexity low while preserving clean seams that could later be extracted into independent services. The API is exposed through Django REST Framework and consumed by a React single-page application. Asynchronous and scheduled work runs on Celery; real-time push runs on Django Channels. PostgreSQL is the system of record, and Redis backs both the Celery broker and the Channels layer.

Component	Technology	Responsibility
Reverse proxy / TLS	nginx	Terminates HTTPS, serves static assets, proxies to the application server
Application server	Gunicorn + Uvicorn (ASGI)	Runs the combined Django / DRF and Channels application
API & domain logic	Django, Django REST Framework	Request handling, serialization, validation, and service-layer business rules
Real-time channel	Django Channels	Pushes balance changes and order-fill events to clients over WebSockets
Task worker	Celery	Executes asynchronous and long-running jobs off the request path

Component	Technology	Responsibility
Scheduler	Celery Beat	Triggers periodic price ticks and monthly statement generation
System of record	PostgreSQL	Durable storage for the double-entry ledger and all domain data
Broker / channel layer	Redis	Celery message broker and Django Channels backing store
Client	React + TypeScript	Single-page UI with live updates, transfer flows, and portfolio charts

Request lifecycle of a synchronous transfer:

1. The React SPA sends an authenticated request (JWT bearer token plus a client-generated idempotency key) over HTTPS.
2. nginx terminates TLS and reverse-proxies the request to the ASGI application server (Uvicorn workers managed by Gunicorn).
3. DRF authenticates the token, applies throttling, and validates the request payload.
4. A service-layer function opens an atomic transaction, locks the affected accounts with row-level locks, and writes balanced journal entries to the ledger.
5. On commit, a domain event is emitted and a Channels consumer pushes the updated balances to the user's open WebSocket connection.
6. The idempotency key and an audit record are persisted, so a retried request returns the original result instead of posting twice.

Asynchronous paths — monthly statement generation and periodic price ticks — are dispatched by Celery Beat to worker processes, decoupling long-running and scheduled work from the request/response cycle.

3. Data Model: The Double-Entry Ledger

The ledger is the spine of the system. Rather than storing a single mutable balance per account, the platform records every financial event as a journal entry containing two or more journal lines whose signed amounts sum to exactly zero — total debits equal total credits. An account's balance is the aggregate of its journal lines, so it can always be independently re-derived and audited; balances are never overwritten in place. This makes whole classes of bugs, such as partial updates and silent balance drift, structurally impossible.

Money is stored as PostgreSQL NUMERIC and manipulated as Python Decimal, eliminating the rounding errors that binary floating-point representation introduces. Each transfer carries a client-supplied idempotency key enforced by a unique constraint, guaranteeing that a retried or double-clicked request posts at most once. All posting logic executes inside a single atomic transaction with row-level locks on the participating accounts, serializing concurrent writes to prevent lost updates and negative-balance races.

The investment domain extends the model with Instrument (ticker metadata), PriceTick (a time series of prices), Order (market or limit, side, and quantity), Fill (executed trades), and Position (holdings tracked with average-cost basis). The security domain adds an append-only AuditLog, per-user TOTP MFA devices, and an encrypted-field abstraction for sensitive columns such as account numbers.

4. Core Features

- *Accounts and real-time transfers.* Customers hold one or more accounts; internal transfers post synchronously through the ledger with full atomicity, idempotency, and balance validation. Completed transfers push live balance updates to connected clients over WebSockets.
- *Investment engine.* A simulated market prices 50+ instruments using geometric Brownian motion, with Celery Beat advancing prices on a fixed interval. Users place market and limit orders; market orders fill at the current tick, while limit orders rest until the simulated price crosses their limit. The engine maintains positions with average-cost basis and computes both realized and unrealized profit and loss.
- *Automated statements.* A scheduled monthly task aggregates each account's ledger activity and renders a formatted PDF statement using ReportLab, retained for the customer to download.
- *Security and authentication.* Authentication uses JSON Web Tokens with short-lived access tokens and rotating refresh tokens, including reuse detection and a logout blocklist. Time-based one-time-password MFA (RFC 6238) is provisioned via QR code. Sensitive fields are encrypted at the application layer, DRF throttling rate-limits abusive clients, and an append-only audit log records every state-changing action with actor, timestamp, and context.
- *Frontend dashboard.* A React and TypeScript single-page application presents accounts, transfer flows, a portfolio view, and price and holdings charts (Recharts), with React Query managing server state and caching and a WebSocket client applying live updates.
- *Reproducible data seeding.* A Faker-driven management command generates the full demonstration dataset — 500+ accounts, 10,000+ transactions, and 1,000+ trades — so the system's headline metrics are deterministic and regenerable on any deployment.

5. Key Design Decisions and Tradeoffs

5.1 Double-entry ledger over a mutable balance column

Storing immutable, balanced journal entries makes every balance reconstructable and auditable, and renders entire classes of correctness bugs structurally impossible. ***Tradeoff:*** *more tables, joins, and write volume than a single balance field, and slightly more complex balance queries — accepted because correctness and auditability are the project's central claims.*

5.2 Fixed-precision DECIMAL over floating point

All money uses NUMERIC and Python Decimal to avoid the binary floating-point rounding errors that are unacceptable in financial arithmetic. ***Tradeoff:*** *marginally slower arithmetic and explicit quantization rules — negligible at this scale.*

5.3 Simulated price engine over a live market-data API

A geometric-Brownian-motion model generates prices in-process, removing any dependence on third-party API keys or rate limits, keeping demonstrations deterministic and offline-capable, and letting the project own the pricing logic. ***Tradeoff:*** *prices are synthetic rather than real; the engine sits behind an adapter interface so a live data source can be substituted later without touching the order or position logic.*

5.4 WebSockets (Django Channels) over REST polling

True server push delivers balance changes and order fills the moment they occur, directly backing the platform's real-time behavior. ***Tradeoff:*** *an ASGI deployment and a Redis channel*

layer add infrastructure over a purely synchronous stack — justified by the materially better real-time experience and the engineering signal of running Channels in production.

5.5 Application-level envelope encryption over database-level encryption

Sensitive fields are encrypted in the application using an AES-GCM data key that is itself wrapped by a separate key-encryption key, mirroring the KMS and HSM envelope pattern used in real payment systems. ***Tradeoff:*** *key management and ciphertext handling live in application code rather than being delegated to the database — chosen deliberately to demonstrate the envelope model and to keep plaintext keys out of the datastore.*

5.6 Celery Beat over system cron

Scheduling statement generation and price ticks inside Celery keeps periodic work in the same codebase and runtime as the rest of the system, with retries, visibility, and horizontal scaling. ***Tradeoff:*** *it requires running broker, worker, and beat processes — already present for asynchronous work, so the marginal cost is low.*

5.7 Modular monolith over microservices

A single deployable with well-bounded Django apps fits the project's scope, simplifies local development and deployment, and avoids premature distributed-systems complexity while leaving clean extraction seams. ***Tradeoff:*** *components share a process and database and scale together — appropriate here, and revisited only if a component's load profile diverges.*

5.8 JWT over server-side sessions

Stateless bearer tokens suit a decoupled SPA, and a potential future mobile client, and avoid server-side session storage. ***Tradeoff:*** *tokens are harder to revoke than sessions; this is mitigated with short access-token lifetimes, refresh-token rotation with reuse detection, and a logout blocklist.*

6. Infrastructure and Deployment

The platform is containerized and deployed on a single AWS EC2 instance via Docker Compose. The composition runs nginx (TLS termination, reverse proxy, and static asset serving), the Django ASGI application (Uvicorn workers managed by Gunicorn), a Celery worker, a Celery Beat scheduler, PostgreSQL, and Redis. TLS certificates are issued and automatically renewed through Let's Encrypt via Certbot. Configuration and secrets are injected as environment variables and are never committed to source control, and database schema changes are applied through Django migrations on each release.

Continuous integration runs on GitHub Actions: every push lints the codebase (ruff and black) and runs the test suite (pytest); on the main branch the workflow additionally builds the production image and deploys it to EC2. Container health checks and a dedicated application health endpoint allow the proxy and orchestration layer to detect and recover unhealthy services.

7. Testing and Reliability

Testing centers on the financial invariants that matter most. Unit and integration tests assert that every journal entry balances to zero, that transfers are idempotent under retry, and that concurrent transfers against the same account cannot produce lost updates or negative balances. Factory-generated fixtures exercise the ledger, the order-matching path, and statement generation. Continuous integration gates every merge on a passing suite, keeping the deployed system in a known-good state.